

# **Reverse Engineering a Mobile App using APKTool**

## **MINI PROJECT REPORT**

*Submitted by*

**Name: Abdul Hanooq Majeed**

**Register No.: 23BTRCC003**

**Name: Sanita**

**Register No.: 23BTRCC034**

*Under the guidance of*

**Dr. Harinee S**

Assistant Professor

*in partial fulfillment of the requirements for the degree*

*of*

**BACHELOR OF TECHNOLOGY**



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**Cyber Security**

**JAIN DEEMED -TO-BE-UNIVERSITY**

**JAKKASANDRA POST, BENGALURU – KANAKAPURA RD, BENGALURU,  
KARNATAKA, 562112. APRIL 2025**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**Cyber Security**

## **BONAFIDE CERTIFICATE**

This is to certify that the project work entitled “**Reverse Engineering a Mobile App using APKTool**” is a bonafide work done by **Abdul Hanooq Majeed [23BTRCC003]** and **Sanita [23BTRCC034]** in partial fulfillment of the requirement, for the award of B.Tech Degree in Computer Science and Engineering by JAIN DEEMED -TO-BE-UNIVERSITY during the academic year 2024-2025.

**PROJECT GUIDE**

**PROGRAM HEAD**

*Submitted for the University Examination held on .....*

**INTERNAL EXAMINER**

**EXTERNAL EXAMINER**

## ACKNOWLEDGEMENT

We are very thankful and grateful to our beloved guide, **Dr. Harinee S** whose great support in valuable advices, suggestions and tremendous help enabled us in completing the final phase of our project. He has been a great source of inspiration to us.

We also sincerely thank our Head of the Department, **Dr. Krishnan Batri**, and our Program Head **Dr. Sunanda Das** whose continuous encouragement and sufficient comments enabled us to complete our project final phase report.

We thank all our **Staff members** who have been by our side always and helped us with our project. We also sincerely thank all the lab technicians for their help as in the course of our project development.

We would also like to extend our sincere gratitude and grateful thanks to our Director **Dr. G Geetha**, Deputy Director **Dr. Krishnan Batri and Dr. Deepak K Sinha** for having extended the Research and Development facilities of the department.

We are grateful to our Chairman **Dr. Chenraj Roychand**. He has been a constant source of inspiration right from the beginning.

We would like to express our faithful and grateful thanks to our Director - Faculty of Engineering and Technology, **Dr. Hariprasad S. A** for his support.

We wish to thank our **family members and friends** for their constant encouragement, constructive criticisms and suggestions with regards to this project review.

Last but not the least; we would like to thank the **ALMIGHTY** for His grace and blessings over us throughout the project.

## ABSTRACT

*In recent years, the widespread use of Android applications has significantly increased the demand for tools that can analyze APK (Android Package) files to identify potential security and privacy risks. As more users rely on mobile applications for personal, financial, and professional purposes, understanding the vulnerabilities inherent in these apps has become essential for maintaining mobile ecosystem security.*

*This project focuses on understanding mobile security threats by creating and analyzing a malicious Android application through reverse engineering techniques. A fake app named “Happening Around the City” was developed using a hybrid mobile development approach with the Ionic framework, JavaScript, and TypeScript. The app simulated malicious behavior by displaying false event information and attempting to extract sensitive user data such as SMS, contacts, clipboard, and storage access. The APK was compiled using Android Studio with Gradle build tools and analyzed using APKTool in a controlled Windows 10 environment with Java JDK 17, Node.js, and Visual Studio Code. The APK file was decompiled, and static code analysis was performed to identify vulnerabilities and unauthorized data access attempts. By working on both the creation and investigation sides, the project provides a comprehensive understanding of mobile application threats and highlights the importance of reverse engineering and potential cyber forensic techniques in securing mobile ecosystems.*

*With the growing number of mobile threats, this project emphasizes the critical need for advanced analysis tools to identify and mitigate security risks in Android applications. The project not only demonstrates the creation of a malicious app but also showcases how reverse engineering can be used to uncover vulnerabilities, ensuring that mobile applications are secure for users.*

**Keywords — Reverse Engineering, APKTool, Android Security, Static Analysis, Hybrid App Development, Ionic Framework, Cyber Forensics, Mobile App Analysis.**

## **LIST OF TABLES**

| <b>TABLE NO.</b> | <b>NAME OF THE TABLE</b> | <b>PAGE NO.</b> |
|------------------|--------------------------|-----------------|
| 5.1              | Comparison Table         | 27-29           |

## **LIST OF FIGURES**

| <b>FIGURE<br/>NO</b> | <b>FIGURE NAME</b> | <b>PAGE NO</b> |
|----------------------|--------------------|----------------|
| 3.1                  | Use Case Diagram   | 16             |
| 3.2                  | Class Diagram      | 17             |

## LIST OF ABBREVIATIONS

**APK** – Android Package (Kit)

**JDK** – Java Development Kit

**SMS** – Short Message Service

**GUI** – Graphical User Interface

**API** – Application Programming Interface

**IDE** – Integrated Development Environment (e.g., Android Studio)

**VM** – Virtual Machine

**XML** – eXtensible Markup Language

**CFG** – Control Flow Graph

**ICFG** – Inter-procedural Control Flow Graph

**UML** – Unified Modeling Language

**OCL** – Object Constraint Language

**SBOM** – Software Bill of Materials

**IoC** – Indicator of Compromise

**NLP** – Natural Language Processing

**APKTool** – Android Application Package Tool (used for decompiling)

**PIN** – Program Instrumentation Tool

**LLM** – Large Language Model

**ICP** – Iterative Closest Point

**DWARF** – Debugging With Attributed Record Formats

**MITRE ATT&CK** – MITRE Adversarial Tactics, Techniques, and Common Knowledge

**OWASP** – Open Web Application Security Project

## TABLE OF CONTENTS

| CHAPTER NO. | TITLE                             | PAGE NO.  |
|-------------|-----------------------------------|-----------|
|             | ABSTRACT                          | IV        |
|             | LIST OF TABLES                    | V         |
|             | LIST OF FIGURES                   | VI        |
|             | LIST OF ABBREVIATIONS             | VII       |
| <b>1.</b>   | <b>INTRODUCTION</b>               | <b>1</b>  |
|             | 1.1 PROJECT OVERVIEW              | 1         |
|             | 1.2 BACKGROUND AND MOTIVATION     | 2         |
|             | 1.3 RESEARCH METHODS              | 2         |
|             | 1.3.1 Investigation Method        | 2         |
|             | 1.3.2 Results                     | 3         |
| <b>2.</b>   | <b>LITERATURE SURVEY</b>          | <b>4</b>  |
| <b>3.</b>   | <b>EXISTING SYSTEM</b>            | <b>11</b> |
|             | 3.1 WORK DONE IN EXISTING SYSTEM  | 11        |
|             | 3.2 ISSUES IN THE EXISTING SYSTEM | 12        |
|             | 3.3 SOLUTION                      | 13        |
|             | 3.4 PROPOSED WORK                 | 13        |
|             | 3.4.1 Problem Definition          | 13        |
|             | 3.5 SYSTEM REQUIREMENTS           | 14        |
|             | 3.5.1 Hardware Requirements       | 14        |
|             | 3.5.2 Software Requirements       | 14        |
|             | 3.5.3 Environment Characteristics | 15        |
|             | 3.5.4 Interface                   | 15        |
|             | 3.6 OBJECT ORIENTED ANALYSIS      | 15        |
|             | 3.6.1 Use case Diagram            | 16        |
|             | 3.6.2 Class Diagram               | 17        |



|           |                                             |           |
|-----------|---------------------------------------------|-----------|
| <b>4</b>  | <b>PROPOSED SYSTEM</b>                      | <b>18</b> |
|           | 4.1 INTRO TO THE PROPOSED SYSTEM            | 18        |
|           | 4.2 PROPOSED SYSTEM – PSEUDOCODE            | 18        |
|           | 4.2.1 System Architecture Overview          | 19        |
|           | 4.2.2 Anomaly Detection Process             | 20        |
| <b>5</b>  | <b>IMPLEMENTATION</b>                       | <b>21</b> |
|           | 5.1 MODULE DESCRIPTION                      | 21        |
|           | 5.2 SCREENSHOTS OF IMPLEMENTATION, RESULT   | 24        |
|           | 5.3 COMPARISON TABLE                        | 27        |
| <b>6.</b> | <b>CONCLUSION &amp; FUTURE ENHANCEMENTS</b> | <b>30</b> |
|           | <b>REFERENCES</b>                           | <b>32</b> |

# CHAPTER 1

## INTRODUCTION

With the rapid growth of mobile applications, the security and privacy of Android apps have become significant concerns, as mobile devices handle sensitive personal, financial, and professional information. To address these challenges, it's crucial to analyze and evaluate the security of these applications. Static code analysis, which involves examining the source or decompiled code without executing the application, is a vital technique for identifying potential vulnerabilities that could be exploited by malicious actors.

This project aims to understand mobile security threats by creating and analyzing a malicious Android application using reverse engineering techniques. A fake app, named "Happening Around the City," was developed using the Ionic framework, JavaScript, and TypeScript. The app simulates malicious behavior by displaying false event information while attempting to extract sensitive user data such as SMS, contacts, clipboard content, and storage access.

The APK file was compiled using Android Studio with Gradle build tools and then analyzed in a controlled environment using APKTool for static code analysis. By decompiling and manually reviewing the code, this project emphasizes the importance of reverse engineering and cyber forensic techniques in identifying vulnerabilities and understanding how malicious apps function. It highlights the need for effective tools and methods to secure mobile ecosystems and protect users from evolving security threats.

### 1.1 PROJECT OVERVIEW

With the rapid increase in the use of mobile applications, Android security has become a major concern. This project focuses on analyzing a fake Android app, "Happening Around the City," which misled users by advertising false event information while secretly extracting sensitive data, such as SMS, contacts, clipboard contents, and storage access. The app was developed using the Ionic Framework, with JavaScript and TypeScript. Through reverse-engineering the APK file using APKTool and conducting static code analysis, we were able to identify malicious code and uncover vulnerabilities embedded within the app's structure.

The investigation was carried out in a controlled Windows 10 environment with tools like APKTool, Java JDK, Node.js, and Visual Studio Code. The goal of this project is to demonstrate the significance of reverse engineering and static analysis in identifying malicious behavior in mobile applications. By decompiling and analyzing the app, we aimed to provide a clearer understanding of how such threats operate and highlight the need for security

measures in mobile app development. This tool offers users the ability to analyze suspicious APKs and make informed decisions, helping to prevent security risks before app installation.

## **1.2 BACKGROUND AND MOTIVATION**

The rapid growth of mobile applications, especially on the Android platform, has significantly changed the way users interact with technology. However, this surge in mobile app usage has also led to an increase in security threats and privacy violations. Android, being the most widely used mobile operating system, has become a primary target for malicious actors seeking to exploit users for illegal activities, such as stealing personal data or committing fraud.

Many Android applications request unnecessary permissions during installation, which, when exploited, can lead to severe security breaches. In some cases, apps that appear legitimate may be designed to gather sensitive information like contacts, SMS, storage, and even microphone and camera access without the user's knowledge. These apps may pose risks by infecting devices with malware or by performing unauthorized actions behind the scenes.

The motivation for this project stems from the need to develop tools and techniques to detect and analyze such malicious behavior in Android apps. By reverse-engineering APK files and performing static code analysis, we can identify vulnerabilities, uncover malicious code, and enhance the security of mobile ecosystems. This project aims to provide a better understanding of how attackers exploit mobile apps and to create a tool that can help users and security professionals analyze suspicious apps before they can cause harm. The goal is to raise awareness about mobile app security and provide a solution for detecting hidden threats in APK files.

## **1.3 RESEARCH METHODS**

This section outlines the research methodology followed during the investigation of the "Happening Around the City" app. The goal was to understand the techniques used by hackers to exploit mobile users through malicious apps and to develop effective detection methods. The research was carried out in three primary phases: analysis of existing reverse engineering tools, academic literature review on app security, and experimental evaluation of malicious code detection methods.

### **1.3.1 Investigation Method**

The investigation began with a thorough review of popular tools used for reverse engineering and analyzing APK files. APKTool was employed as the primary tool for decompiling APKs and examining the app's resources and manifest files, allowing us to gain a better understanding of the app's structure and the way it interacts with the Android operating system. This provided

valuable insights into the underlying code, revealing how the app was built and its various components. To complement this, Apktool-bat, a batch script, was used to automate the process of APK decompiling, making the workflow more efficient and reducing manual effort. Alongside these tools, static code analysis was applied to inspect the JavaScript files embedded within the app. The analysis focused on identifying suspicious code patterns that could be linked to data extraction activities, such as accessing sensitive information like SMS messages, contacts, clipboard contents, and storage. These are all potential vectors through which malicious code can operate without the user's knowledge. Additionally, we extended our research to investigate hybrid mobile applications, particularly those built using frameworks like Ionic, which combine JavaScript, TypeScript, and native Android components to create versatile apps. Understanding the development process behind these hybrid apps, which can include both web technologies and native code, was crucial in identifying potential security vulnerabilities. By dissecting the structure of these APK files and understanding the tools used in their creation, we were able to uncover the techniques hackers employ to insert malicious code and exploit user data, often without triggering alarms or raising suspicion among users.

### **1.3.2 Results**

The investigation into the "Happening Around the City" app revealed key findings about its malicious intent. Static code analysis of the JavaScript files uncovered suspicious behavior, with keywords like SMS, contacts, clipboard, and storage, indicating the app's ability to secretly extract sensitive user data. This confirmed that the app was designed for unauthorized data collection without user consent.

Further examination of the app's structure showed it was built using the Ionic Framework, which allowed for the integration of both web-based and native components. This hybrid development approach, while convenient, increased the attack surface and made it easier to inject harmful code. APKTool and Apktool-bat were crucial in decompiling the app, revealing hidden functionalities intended to exploit user data.

The investigation highlighted the gap in user awareness regarding the security risks posed by seemingly harmless apps. Many users were drawn to the app because of its "zero-fee" promise, unaware of its malicious nature. This reinforced the need for better tools and education to help users protect themselves from such threats. Overall, the results emphasized the risks associated with hybrid app development and the importance of improved detection methods to combat malicious activities.

## **CHAPTER 2**

### **LITERATURE SURVEY**

#### **1. An Investigation of Online Reverse Engineering Community Discussions in the Context of Ghidra**

**Authors:** Daniel Votipka, Mary Nicole Punzalan, Seth M. Rabin, Yla Tausczik, Michelle L. Mazurek

**Date of Publication:** 04 November 2021

This research paper investigates how the reverse engineering community adopts and shares knowledge about new tools, focusing on NSA's Ghidra framework. The methodology involved analyzing 1,590 discussions by 688 users across Twitter, Reddit, and StackExchange to understand community dynamics, feature interests, and support behaviors. The results showed a strong interest in Ghidra's customization features and highlighted that users typically acted either as knowledge providers or seekers, with limited collaborative problem-solving. Twitter was primarily used for announcements, StackExchange for technical support, and Reddit as a blend of both. A major strength of the study is its comprehensive analysis across multiple platforms, offering insights into tool adoption behavior. However, a limitation is the lack of deeper engagement analysis beyond initial posts. The paper suggests enhancing tool development and fostering more active community involvement as future directions.

#### **2. Exploring IoT Device Vulnerabilities Through Malware Analysis and Reverse Engineering**

**Authors:** R. Tamilkodi, V. Bala Sankar, Shrija Madhu, L. Revathi, V. Sai Guna Srikar, Ernagula Ajay

**Date of Publication:** 31 December 2024

This research project focuses on analyzing IoT device vulnerabilities by reverse engineering malware targeting these systems. The methodology combines static and dynamic analysis, using tools like IDA Pro, Ghidra, and Wireshark to disassemble malware, trace behaviors, and analyze communications. The study identifies common vulnerabilities such as weak authentication, poor input validation, and inadequate firmware security. Results show how malware exploits these flaws and reveal techniques like payload delivery and persistence mechanisms. A key strength of this research is its comprehensive use of analysis tools and focus on real-world malware behavior. However, it may be limited by the scope of malware samples analyzed. The study offers practical recommendations for improving IoT security through better detection strategies and secure device design.

### **3. Revisiting Deep Learning for Variable Type Recovery**

**Authors:** Kevin Cao, Kevin Leach

**Date of Publication:** 13 July 2023

This research builds upon Chen et al.'s DIRTY model, a Transformer-based architecture designed to enhance decompiled code by predicting variable names and types using decompiler outputs. The original DIRTY model showed high accuracy on Hex-Rays decompiler outputs. This study extends that work by retraining the model using data from the open-source Ghidra decompiler, addressing previous concerns about Ghidra's complexity in parsing DWARF symbols. The methodology involves straightforward parsing of Ghidra-generated variable data to replicate DIRTY's performance. Results indicate comparable retyping accuracy to Hex-Rays, validating Ghidra's potential for similar research. A strength of this study is its promotion of accessible, open-source tooling for reverse engineering. However, a limitation is the reliance on parsing techniques that may not generalize across all binary formats. Overall, the research encourages broader use of Ghidra in machine learning-based reverse engineering tasks.

### **4. AI-Powered Ransomware Detection Framework**

**Authors:** Subash Poudyal, Dipankar Dasgupta

**Date of Publication:** 05 January 2021

This research proposes an AI-based ransomware detection framework named AIRaD, aimed at improving detection accuracy through a combination of static and dynamic analysis techniques. The methodology involves dynamic binary instrumentation using the PIN tool, function call tracing with Cuckoo sandbox, and analysis with Ghidra. Features are extracted at DLL, function call, and assembly levels, then processed using NLP and association rule mining. These features are fed into machine learning classifiers, where Support Vector Machine and Adaboost with J48 achieved the highest accuracy of 99.54% and a low false-positive rate of 0.005. A major strength of this work is its multi-level, hybrid approach combining traditional analysis with AI techniques. However, it may be limited by the complexity and resource intensity of the toolchain. The study contributes an effective and robust model for detecting evolving ransomware threats with high precision.

### **5. Optimizing Decompiler Output by Eliminating Redundant Data Flow in Self-Recursive Inlining**

**Authors:** Runze Zhang, Ying Cao, Ruigang Liang, Peiwei Hu, Kai Chen

**Date of Publication:** 19 December 2024

This research introduces ERASE, a novel approach designed to reverse the self-recursive inlining optimization that often reduces the readability of decompiled binary code. The methodology focuses on improving the clarity of decompiled functions by undoing complex transformations introduced by compiler optimizations. ERASE is evaluated against popular decompilers Ghidra and Hex-Rays, with results showing its output is significantly more compact 78.4% and 88.9% fewer lines than Ghidra and Hex-Rays, respectively. Additionally, reverse engineers required 88.5% less time to analyze ERASE's output compared to Ghidra and 90.4% less time compared to Hex-Rays, with analysis accuracy increasing by 2.75 times. A key strength of this work is its focus on enhancing human readability and efficiency in reverse engineering. However, its applicability may be limited to scenarios involving self-recursive inlining. Overall, ERASE offers a valuable advancement for simplifying complex decompiled code and aiding security analysts in binary analysis tasks.

## **6. GrizzlyBay: Retroactive SBOM with Automated Identification for Legacy Binaries**

**Authors:** Junghwan "John" Rhee, Fei Zuo, Cody Tompkins, Jeehyun Oh, Ying Ryn Choe

**Date of Publication:** 06 December 2024

This research presents GrizzlyBay, an innovative approach for generating Software Bill of Materials (SBOM) for legacy native binaries, which typically lack metadata. Unlike traditional tools limited to source code or package-managed software, GrizzlyBay uses locality-sensitive hashing of abstract instructions and program structures to automatically identify software names, versions, and associated vulnerabilities. The methodology emphasizes speed, reliability, and compact representation. When evaluated, GrizzlyBay achieved accuracy comparable to Ghidra but outperformed it in terms of processing speed and output conciseness. A major strength of this work is its ability to enhance software transparency and security in legacy systems. However, it may be limited by the diversity and complexity of binary formats in the wild. Overall, the approach provides an effective solution for SBOM generation in environments lacking source code access.

## **7. DroidCC: A Scalable Clone Detection Approach for Android Applications to Detect Similarity at Source Code Level**

**Authors:** Junaid Akram, Zhendong Shi, Majid Mumtaz, Ping Luo

**Date of Publication:** 22 June 2018

This paper introduces DroidCC, a novel clone detection approach tailored for Android applications, aimed at improving code security and maintainability. DroidCC analyzes APK source code to identify type-1, type-2, and type-3 clones, as well as suspicious code fragments

that may indicate malware spreading. The methodology includes building a prototype and evaluating it on a large dataset of nearly 30,000 top-rated Android apps. DroidCC effectively detects both full and partial similarities between apps, highlighting potential risks of reused or malicious code. Results show high recall and precision, proving the approach's efficiency and accuracy. A major strength of DroidCC is its ability to handle various clone types and large-scale data. However, the approach may face challenges with heavily obfuscated code or encrypted payloads. Overall, DroidCC significantly enhances Android app analysis by supporting early detection of code clones and potential vulnerabilities.

## **8. Automated Threat Hunting, Detection, and Threat Actor Profiling Using TIRA**

**Authors:** Semi Yulianto, Gerard Nathaniel Cac Ngo

**Date of Publication:** 19 November 2024

The study presents TIRA, a lightweight system for cybersecurity that operates efficiently in low-computational environments. Methodologically, TIRA leverages the OWASP Top 10 and MITRE ATT&CK frameworks to enhance log parsing, detect threat patterns, and build attacker profiles. It integrates advanced detection rules and framework mapping to aid non-expert users in identifying and addressing cyber threats effectively. Results show that TIRA analyzed a 1.1 GB log file with over 11 million entries in under an hour, compared to 24 hours for manual analysis, achieving a 95.83% reduction in threat detection and response time. Advantages of TIRA include its resource efficiency, improved incident handling, and the ability to provide actionable threat intelligence, especially for organizations with limited resources. It also encourages industry collaboration and organizational preparedness. However, a potential disadvantage may be its dependency on predefined frameworks, which could limit detection of novel or highly sophisticated threats not covered by OWASP or MITRE standards.

## **9. The Making of Indicator of Compromise using Malware Reverse Engineering Techniques**

**Authors:** Bio Akram, Dion Ogi

**Date of Publication:** 04 January 2021

Malware threats often remain undetected initially because attackers skillfully camouflage their presence within systems, with users only realizing the damage after their devices malfunction. To evade detection mechanisms, malware authors frequently use packers to obscure malicious content. Malware analysis serves as a crucial method to understand and investigate such threats, with reverse engineering being a particularly effective technique for analyzing the behavior and structure of malware, especially those employing packing methods. This approach helps uncover the internal workings of the malware and aids in crafting Indicators of Compromise



(IoCs). These IoCs are then translated into YARA rules, which are pattern matching rules designed for identifying malware based on known characteristics. Thus, reverse engineering becomes a vital step in creating effective YARA rules for enhanced malware detection.

#### **10. Reverse Engineering-A Method for Analyzing Malicious Code Behavior**

**Authors:** Vivek Bhardwaj, Vinay Kukreja, Chetan Sharma, Isha Kansal, Renu Popali

**Date of Publication:** 07 February 2022

The paper utilizes both static and dynamic analysis techniques, along with software reverse engineering, to analyze infected binary files without requiring access to source code. Static analysis involves examining the malware code without execution, while dynamic analysis observes the malware's behavior during runtime. The combined use of these techniques enables a comprehensive understanding of malware characteristics and their impact on systems. The results demonstrate effective identification of malware behavior, making the approach suitable for real-world scenarios. Among the advantages are its applicability to unknown binaries and its ability to extract meaningful insights using dual analysis methods. However, limitations include the inability of static analysis to detect runtime behavior, the potential for malware to evade detection in dynamic environments, and the requirement for specialized expertise and time-consuming processes.

#### **11. Point cloud data fitting-based reverse engineering technique for digital model updating of assembly structure**

**Authors:** Jihong Yan, Huaqiu Ding, Jinsong Zhang, Pengxiang Wang

**Date of Publication:** 20 December 2021

The methodology involves using virtual assembly technology to detect potential issues in the structural assembly process, helping reduce the high costs and complexity of mechanical equipment development. A general approach is proposed to reconstruct and update assembly geometric models by integrating parametric modeling, synchronous modeling, and reverse engineering. A 3D-printed free-form surface block is used as a case study, with laser scanning generating point cloud data. A k-d tree is built to define topological relationships, and the Iterative Closest Point (ICP) algorithm is used alongside the k-d tree search strategy to align point clouds from various angles. The combined data is projected onto a two-dimensional plane for triangulation and surface reconstruction. The results show accurate model alignment and improved assembly accuracy. Advantages of this method include better detection of assembly issues, improved model accuracy, and reduced R&D costs, while drawbacks include the complexity of integrating modeling techniques and high computational demands.

## **12. REDBUL: An Online System for Reverse Engineering of Relational Databases**

**Authors:** Drazen Brdjanin, Dragana Vukovic, Goran Banjac, Aleksander Kelec, Igor Dujlovic, Nikola Obradovic

**Date of Publication:** 11 September 2020

The methodology presented in the paper involves the development of an online system named REDBUL, designed for reverse engineering relational databases. REDBUL automatically extracts the schema from existing databases and visualizes it as a standardized UML class diagram within a web browser, offering an intuitive and accessible means for database designers to comprehend and analyze database structures. The system currently supports two relational database management systems: Microsoft SQL Server and MySQL, with the demonstration primarily focused on MySQL. The results highlight REDBUL's ability to streamline and automate the schema extraction and visualization process, thereby enhancing database documentation and understanding. Key advantages of the system include the automation of the reverse engineering process, compatibility with multiple database systems, and ease of use through a web-based interface. However, the system's limitations include its current support for only two database management systems and potential challenges in handling very large or highly complex database schemas.

## **13. Advanced Malware Analysis Methods: Behaviour-Based Detection and Reverse Engineering**

**Authors:** Nur Widiyasono, Siti Rahayu Selamat, Randi Rizal, Abdul Fidayan, Sri Rochani Mulyani, Slamet Risnato.

**Date of Publication:** 06 February 2025

The methodology in this research focuses on analyzing and mitigating the Agent Tesla malware through three primary approaches: behavior-based detection using virtual machines, behavior-based detection utilizing sandbox environments, and reverse engineering. Behavior-based detection involves monitoring the malware's activities within a controlled environment to observe its operational behavior, while reverse engineering is employed to extract and analyze the underlying code structure of the malware. The findings reveal that Agent Tesla is capable of exfiltrating sensitive information, including usernames, email credentials, and stored passwords from web browsers such as Microsoft Edge, Mozilla Firefox, and Google Chrome. The malware infiltrates systems, accesses local data, and stores the extracted information. Among the evaluated methods, behavior-based detection via sandboxing demonstrated the highest effectiveness, capturing 3877 actions, an increase of 185 percent compared to reverse engineering, thereby underscoring its enhanced ability to reveal malware behavior. The advantages of this approach include comprehensive detection of malicious activities and deeper behavioral insights, particularly through sandbox analysis. However, its limitations include the

complexity and high resource requirements associated with sandbox environments, as well as the restricted capability of reverse engineering in capturing dynamic malware actions.

#### **14. Enhancing Model-Driven Reverse Engineering Using Machine Learning**

**Authors:** Hanan Abdulwahab Siala

**Date of Publication:** 20 June 2024

The methodology presented in this research introduces a novel Model-Driven Reverse Engineering (MDRE) approach aimed at modernizing legacy systems through the extraction of their architecture, functionality, and business rules. This process leverages Large Language Models (LLMs) to analyze source code and automatically generate Unified Modeling Language (UML) diagrams and Object Constraint Language (OCL) representations, thereby facilitating a deeper understanding of the structural and behavioral aspects of legacy systems. The results indicate that incorporating LLMs into reverse engineering processes can significantly improve the accuracy and efficiency of deriving meaningful models from outdated codebases. The advantages of this approach include the automation of complex analytical tasks, enhanced comprehension of legacy systems, and support for effective system modernization. However, potential limitations involve dependency on the quality and training of the LLMs, the risk of inaccuracies in the generated models, and challenges associated with processing extensive or poorly documented legacy code.

#### **15. FineObfuscator: Defeating Reverse Engineering Attacks with Context-sensitive and Cost-efficient Obfuscation for Android Apps**

**Authors:** Balakrishnan Dharmalingam, Anyi Liu, Subra Ganesan, Sankardas Roy

**Date of Publication:** 27 July 2022

The methodology presented in this research involves the development of FineObfuscator, a context-sensitive obfuscation system designed to mitigate reverse engineering attacks (REAs) by selectively obfuscating only critical components within a software package. The system employs static analysis using Control Flow Graphs (CFGs) and inter-procedural control flow graphs (ICFGs) to identify sensitive code segments. It then applies targeted obfuscation based on a set of developer-defined obfuscation policies, allowing for a customized and fine-grained security strategy. The results of the evaluation, conducted on widely referenced Android repositories, indicate that FineObfuscator effectively hinders reverse engineering attempts while requiring significantly fewer computational resources compared to existing full-scale obfuscation tools. This approach enhances efficiency and maintains performance by avoiding unnecessary obfuscation of non-critical code. However, the system's reliance on accurate identification of critical components introduces challenges in ensuring comprehensive protection, and the selective nature of the obfuscation process may leave some portions of the software relatively more exposed.

## **CHAPTER 3**

### **EXISTING SYSTEM**

The existing system involves a comprehensive process for analyzing and reverse-engineering suspicious mobile applications, specifically focusing on APK (Android Package) files. This system utilizes tools like Apktool to decompile APK files into a readable format, enabling experts to examine the app's structure, source code, and resources such as images and configuration data. Static code analysis is performed on the decompiled files to search for suspicious patterns, particularly targeting keywords like sms, contacts, clipboard, and storage, which could indicate malicious data extraction or unauthorized access. The Android Manifest file is carefully inspected for any abnormal permissions, such as those requesting access to sensitive user information. The system is built on a robust environment, utilizing tools like Java JDK, Node.js, Ionic Framework, Android Studio, and Apktool, running on a Windows 10 system with sufficient resources for efficient analysis. The workflow is divided into three logical modules: Apktool-jar for decompilation, Apktool.bat to trigger the process, and ApkAnalyzer, a manual step that may be automated in the future. This existing system is primarily used for mobile app security assessments, bug detection, and ensuring compliance with data privacy regulations, allowing cybersecurity professionals to identify and mitigate threats, debug applications, and safeguard mobile users from malicious activities.

#### **3.1 WORK DONE IN EXISTING SYSTEM**

Under the existing system, a comprehensive analysis was carried out to investigate the suspicious mobile app "HAPPENING AROUND THE CITY" for malicious activities. The process began by downloading the app and extracting the APK file for further examination. Cyber analysts used the Apktool to decompile the APK, allowing them to break down the app into its individual components, such as source code and configuration files. Once decompiled, the app's structure was carefully analyzed to identify the underlying technology used in its development, specifically recognizing it as a hybrid app built on the Ionic Framework using JavaScript.

Static code analysis was then performed on the decompiled files, with a particular focus on identifying any suspicious or malicious code. Analysts searched for specific keywords like sms, contacts, clipboard, and storage within the JavaScript files, which are commonly associated with unauthorized access to sensitive user data. Additionally, the AndroidManifest.xml file was inspected for any permissions that could potentially exploit mobile user information.

In terms of tools and environment setup, the system made use of a Windows 10 operating system with a configuration of 16GB RAM, 8 CPUs, and 500GB of storage, providing a solid foundation for efficient analysis. Key tools such as Apktool, Java JDK 17, Node.js, and Android Studio were employed to carry out the decompilation and debugging processes. Apktool was used for the decompilation of the APK file, while other tools like Visual Studio Code helped in writing scripts and analyzing the code.

The overall work done under the existing system enabled the identification of malicious activities within the app, confirming that it was indeed a fake app with hidden functions designed to illegally access user information. The system's reverse engineering process helped uncover the app's true intentions, ultimately leading to the conclusion that it was malicious and posed a significant security risk to users.

## **3.2 ISSUES IN THE EXISTING SYSTEM**

The existing system, while effective in its approach, faces several limitations that hinder its full automation and scalability. One of the primary issues is that the system primarily relies on static code analysis, which can only detect known vulnerabilities and malicious code based on pre-defined patterns. This method is not dynamic and may miss new, sophisticated attack vectors that evolve over time.

Additionally, the system is semi-automatic, as it requires manual intervention at several stages of the analysis process. While Apktool is used to decompile the APK files, this is just the first step in the workflow. After decompilation, the actual analysis of the app's code, manifest files, and suspicious keywords must still be carried out manually by the cyber analysts. This manual step introduces delays and can be error-prone, especially when dealing with large volumes of apps or sophisticated malicious code. The lack of full automation means that human expertise is still necessary to interpret findings and identify threats, which can lead to inconsistencies in analysis.

Another issue is that static analysis alone is limited in detecting runtime threats or hidden malicious behaviors that only become apparent during the execution of the app. Since the system does not perform dynamic analysis or behavioral monitoring, it may not identify activities such as data exfiltration, live command and control communications, or real-time interactions with external servers.

Moreover, the current system lacks an automated reporting mechanism to consolidate findings. Analysts must manually document their results, which can be time-consuming and may lead to incomplete or delayed reports.

Finally, as the system depends on certain tools like Apktool for decompilation and static analysis, it may face challenges in handling encrypted or obfuscated APK files, which are increasingly common in malicious apps. These tools may not always be effective in unpacking such apps, potentially leaving some malicious code undetected.

Overall, the existing system would benefit from a more automated, dynamic, and scalable approach that can handle both static and dynamic analysis, as well as provide real-time monitoring and reporting capabilities.

### **3.3 SOLUTION**

The solution to address the issues in the existing system involves transitioning towards a fully automated, dynamic, and scalable approach to APK analysis. First, automating the APK decompilation process will eliminate the need for manual intervention and speed up the workflow. To complement static code analysis, integrating dynamic analysis tools like Frida or Xposed will allow for real-time observation of an app's behavior during execution, enabling the detection of hidden malicious activities such as unauthorized data exfiltration or communication with external servers. Additionally, incorporating machine learning models can help identify previously unknown threats by analyzing patterns in the app's code, while AI-powered tools can automatically flag suspicious behaviors. The system will also benefit from an automated reporting mechanism, generating detailed reports and visualizations to assist analysts in making quick, data-driven decisions. To handle obfuscated or encrypted APK files, advanced deobfuscation techniques and cryptographic analysis will be employed. Finally, a cloud-based platform can be implemented for scalable, efficient analysis of multiple apps, integrating with third-party tools, AI models, and real-time monitoring dashboards. This solution will provide a more robust, accurate, and timely system for detecting and mitigating security threats in mobile applications.

### **3.4 PROPOSED WORK**

This project focuses on enhancing the cyber investigation process for detecting and analyzing malicious mobile applications distributed through fake event promotions. Traditional analysis methods rely mainly on surface-level inspections, often missing deeper vulnerabilities hidden within hybrid frameworks like Ionic. The proposed system aims to build a structured approach by integrating reverse engineering, decompilation, static code analysis, and malware behavior inspection. By utilizing tools such as Apktool, static analysis scripts, and manual inspection techniques, the system enables analysts to dissect APK files, investigate file structures, search for sensitive API usage, and trace hidden malicious code injected into JavaScript resources. A key feature of this approach is the ability to pinpoint unauthorized access to sensitive user data like SMS, contacts, clipboard, and storage. Additionally, the system enhances threat detection by applying targeted keyword analysis and folder-level exploration. The goal is to produce a detailed report highlighting observed vulnerabilities, potential backdoors, and associated risks. This ensures a transparent evaluation for end-users and law enforcement agencies, thereby strengthening mobile app security practices against social engineering attacks

.

#### **3.4.1 Problem Definition**

In today's mobile-first world, users increasingly trust mobile applications for accessing information, attending events, and managing daily activities. However, malicious actors exploit this trust by distributing fake applications disguised under attractive offers such as "free

events." These applications, once installed, secretly access sensitive personal data without user consent. Existing APK analysis methods are often not designed to thoroughly inspect hybrid mobile applications, which utilize JavaScript and web technologies embedded inside Android shells. Current tools mostly extract metadata and permissions but fail to detect malicious behaviors hidden inside hybrid app layers. Furthermore, the lack of systematic investigation methods for hybrid frameworks leaves cybersecurity teams struggling to identify threats proactively. Therefore, there is a critical need for a specialized approach capable of decompiling, inspecting, and analyzing hybrid mobile applications deeply. A comprehensive system that performs detailed folder analysis, JavaScript file inspection, API usage tracing, and behavior detection will bridge this gap, making mobile app investigations more accurate and reliable.

## **3.5 SYSTEM REQUIREMENTS**

To efficiently execute the APK reverse engineering and analysis workflow, specific hardware and software prerequisites are necessary. The system should be capable of handling decompilation processes, static code analysis, and large file handling without performance bottlenecks.

### **3.5.1 Hardware Requirements**

The system requirements for effective APK analysis and decompilation involve a processor such as the Intel Core i5 (8th Gen or above) or an equivalent, ensuring smooth performance during the process. A minimum of 8GB of RAM is required, although 16GB is highly recommended for better efficiency and performance, especially when handling larger APK files and running decompilation tools. In terms of storage, at least 50GB of free space is necessary to accommodate APK files, decompiled resources, and detailed analysis reports. The operating system should be either Windows 10/11 (64-bit) or Ubuntu 20.04 or higher, providing a stable and compatible environment for running APK analysis software. A reliable and stable internet connection is crucial for keeping tools up-to-date and accessing threat intelligence databases, which are integral for security analysis. While not mandatory, an SSD is strongly recommended, as it significantly boosts read/write operations during APK extraction, resource browsing, and overall system responsiveness, leading to a smoother and more efficient analysis experience.

### **3.5.2 Software Requirements**

The recommended environment for effective APK reverse engineering and analysis includes a robust operating system setup, preferably Windows 10/11 or a Linux distribution such as Ubuntu 20.04 or higher, to ensure compatibility and stability with various tools. Key reverse engineering tools needed are Apktool, which is essential for decompiling APK files and extracting resources, and Visual Studio Code, which serves as a powerful platform for static code analysis, offering flexibility through extensions and integrations. In terms of development

tools, it is important to have Java JDK 17 or above installed to handle Java-based applications, along with Node.js and npm to facilitate the analysis of hybrid mobile applications when necessary. Supporting tools are equally important, in future including a Python 3.8+ environment for running automation scripts that can streamline repetitive tasks, and static code analyzers designed to assist in tracing important keywords, searching files efficiently, and identifying potential vulnerabilities within the code. An Android Emulator, such as the one provided by Android Studio, can be optionally used for dynamic behavior observation, allowing analysts to monitor how an application interacts within a simulated Android environment. Finally, having administrator privileges is crucial, as it enables the proper installation and execution of diagnostic and security tools without restrictions, ensuring a smooth, uninterrupted reverse engineering and analysis workflow.

### **3.5.3 Environment Characteristics**

The investigation system must be set up in a secure and isolated environment, preferably a sandbox or virtual machine, to prevent accidental exposure to malicious code. It should support installation of reverse engineering tools, debugging environments, and have enough memory and processing power to handle large hybrid apps. Internet access must be monitored and secured to prevent unauthorized outbound connections during analysis.

### **3.5.4 Interface**

Although this project primarily follows a manual investigation method, future enhancements may introduce a simple dashboard to upload APK files and generate static reports. For the current stage, analysis outputs will be recorded manually in a structured report format, highlighting key findings, file paths, observed malicious behaviors, and suggested remediation measures. Screenshots of tool outputs and decompiled code segments will be included for documentation and evidence.

## **3.6 OBJECT ORIENTED ANALYSIS**

The object-oriented analysis of the APK analysis process, as depicted in the diagrams, identifies distinct functional components and their interactions as classes and objects. Each task such as downloading the APK, decompiling it using tools like APKTool, extracting the manifest and resources, and analyzing decompiled Java code can be represented as separate classes with specific responsibilities and behaviors. These classes interact collaboratively to identify potentially malicious or vulnerable code patterns and generate a comprehensive analysis report. The sequential flow and modular design suggest the use of encapsulation and responsibility-driven design, where each component class is accountable for a single aspect of the analysis process. This approach enhances maintainability, scalability, and clarity of the analysis system, allowing future extensions such as automated threat detection or integration with threat intelligence feeds.



### 3.6.1 Use Case Diagram

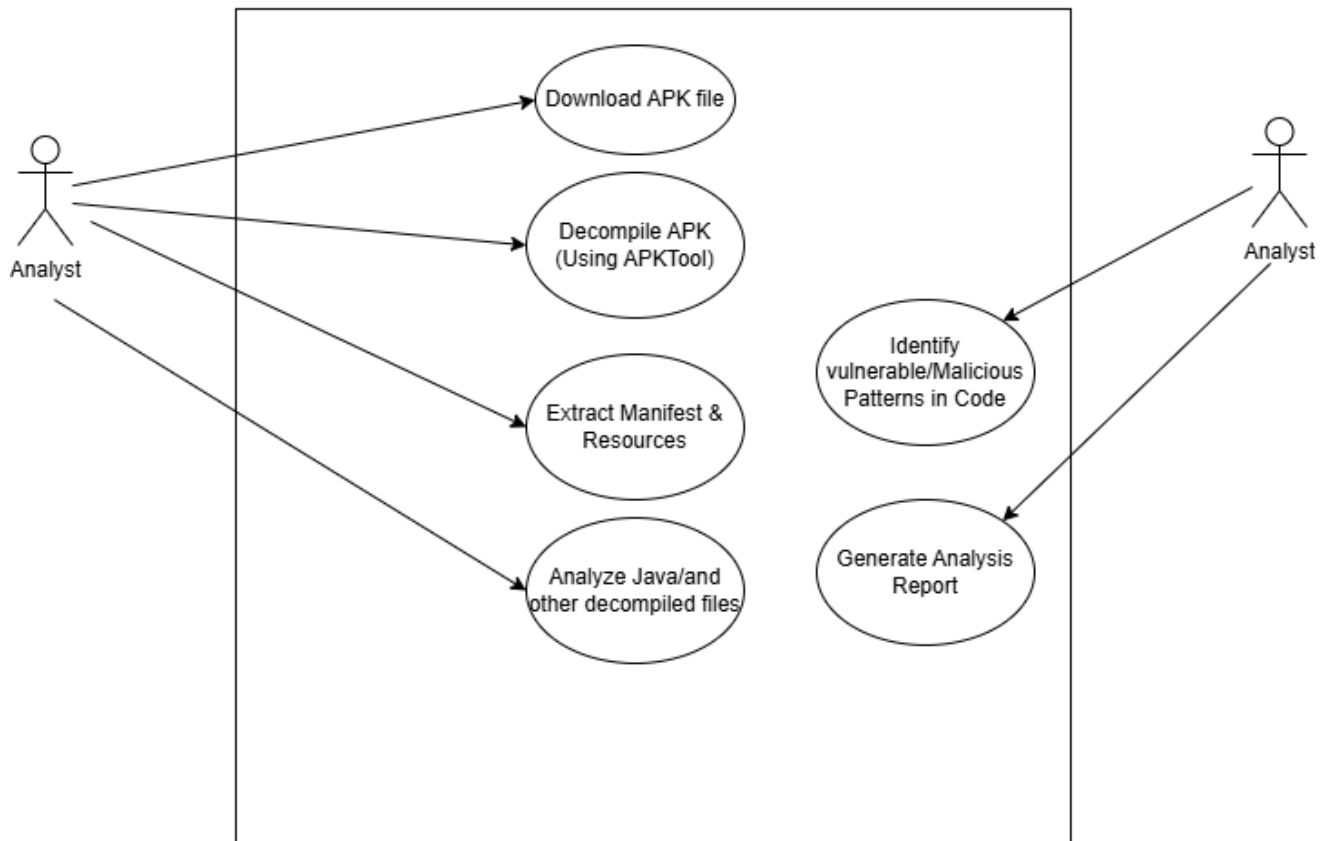


Fig 3.1 Use Case Diagram

### 3.6.2 Class diagram

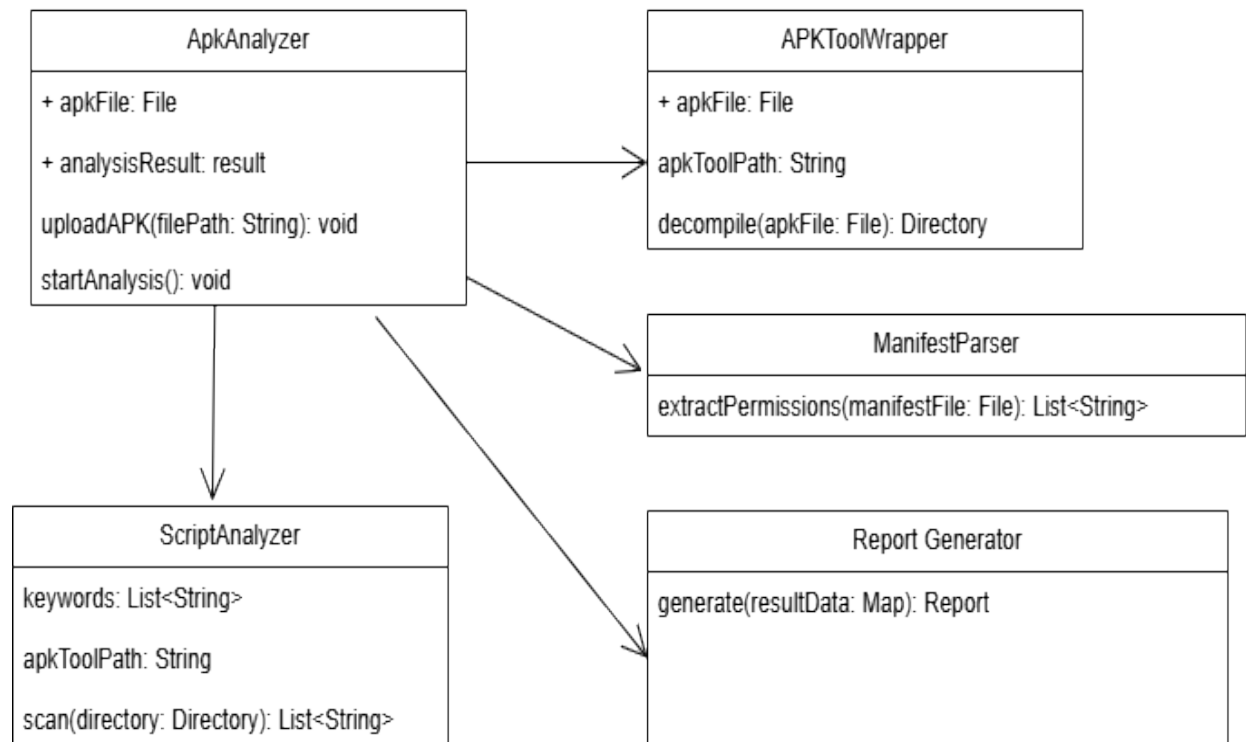


Fig 3.2 Class Diagram

## **CHAPTER 4**

### **PROPOSED SYSTEM**

The proposed system is designed to improve the detection and investigation of fake Android applications that are used to steal user information. It focuses on analyzing hybrid apps, especially those built using frameworks like Ionic, which often embed malicious scripts in JavaScript files. The system begins by collecting the APK file either from the mobile device or an emulator. Using tools like Apktool, the APK is decompiled to extract its source code, manifest file, and internal structure. This allows analysts to explore hidden resources and identify suspicious activities. Static analysis techniques are applied to search for sensitive API calls and keywords such as "sms," "contacts," "clipboard," and "storage." This helps detect unauthorized access attempts and data leakage points within the app. The investigation pays special attention to files like AndroidManifest.xml and JavaScript resources where malicious intent can be hidden. Manual code walkthroughs support tracing data flow and identifying hidden backdoors or unauthorized network connections. Unlike traditional APK analyzers that focus only on metadata, this system provides a deeper view into the app's behavior, even when it is disguised as a legitimate application. The analysis results are recorded in a structured report, including screenshots, file paths, observed threats, and a final conclusion about the app's safety. Although the system currently relies on manual and semi-automated techniques, it can be enhanced in the future by adding automated scripts for faster keyword searching and behavior simulation. Overall, this project offers a simple but effective method to uncover and document malicious activities in hybrid Android apps, helping improve mobile app security assessments.

#### **4.2 PROPOSED SYSTEM– PSEUDOCODE**

Start

Input: APK file (e.g., app.apk)

Step 1: Initialize Apktool

- Load necessary resources (framework files, decoding libraries)

Step 2: Parse the APK file

- Extract AndroidManifest.xml
- Extract resources (res/ folder)

- Extract compiled code (classes.dex)

#### Step 3: Decode resources

- Decode binary XML files into human-readable XML
- Decode resource files (images, layouts, etc.)

#### Step 4: Reconstruct the folder structure

- Create output directories
- Place decoded files into appropriate folders

#### Step 5: Generate project-like source code

- Organize files into folders (res/, smali/, assets/)
- Prepare AndroidManifest.xml for easy viewing

#### Step 6: Output

- Save all decompiled files into a new folder
- Provide ready-to-analyze code and resources

End

### 4.2.1 System Architecture Overview

The architecture of the proposed APK analysis system follows a structured and systematic flow designed to detect malicious activities hidden inside suspicious mobile applications. The process begins when users download an application from sources like the Google Play Store onto their Android devices. If an app is suspected of malicious behavior, its APK file is extracted from the mobile device and transferred to a secure computer system for detailed investigation.

On the computer, the APK file is subjected to decompilation using tools such as Apktool. Apktool processes the input APK and generates an output folder containing the application's internal files, resources, and source structure. This enables analysts to examine critical components like the AndroidManifest.xml file, JavaScript scripts, and other embedded resources for signs of malicious activity.

Once decompiled, the APK contents are fed into the APK Analyzer module. The APK Analyzer systematically reviews the output folder by focusing on sensitive areas such as permission

usage, hardcoded URLs, access to device resources (contacts, SMS, storage, clipboard), and abnormal API calls. Keyword-based searches and static analysis methods are applied to identify vulnerabilities or backdoors.

The system architecture ensures that the analysis is performed in a secure environment with a clear separation between input APK handling, decompilation, and analysis phases. By following this multi-step process, investigators can detect malicious intent, gather evidence, and generate a complete security report, improving the identification and prevention of mobile app-based threats.

The overall functional flow, as depicted in the architecture diagram, highlights the transition of the APK from mobile device to analysis environment, ensuring a logical and secure approach to reverse engineering and threat detection.

#### **4.2.2 Anomaly Detection Process**

The anomaly detection process was initiated after multiple mobile users lodged complaints with the cyber cell, reporting that the app "HAPPENING AROUND THE CITY," which promised free access to local events, was fake and potentially malicious. Acting promptly, the cyber analyst team downloaded the suspicious application and extracted its APK file, transferring it securely to a diagnosis machine configured with the necessary tools for forensic analysis. The first step involved decompiling the APK using the tool Apktool through the command `apktool d app.apk`, which unpacked the internal structure of the application for further inspection. Upon reviewing the directory structure, analysts determined that the app was developed as a hybrid mobile application built on the Ionic framework, primarily using JavaScript and TypeScript programming languages. Recognizing the possibility of malicious intent, the team proceeded with a static code analysis, scanning all decompiled JavaScript files for critical keywords such as `sms`, `contacts`, `clipboard`, and `storage` to identify any unauthorized attempts to access sensitive user data. Special attention was given to the `AndroidManifest.xml` file, where permissions requested by the application were scrutinized to detect any overreach that could compromise user security. Filtered JavaScript files were manually inspected to locate any hidden malicious functions or data exfiltration mechanisms. The detailed investigation revealed that the app contained embedded code capable of accessing users' personal information without their consent, confirming the presence of serious security violations. This malicious behavior indicated that the application was specifically engineered to exploit users by collecting sensitive information under the guise of promoting fake events. Based on comprehensive static and manual code analysis, it was conclusively determined that the app was fraudulent and deployed for illegal activities, leading to its categorization as a severe cybersecurity threat.

## CHAPTER 5

### IMPLEMENTATION

The implementation of the APK Analyzer focuses on providing users with an efficient tool to reverse engineer Android APK files using APKTool. This tool is primarily command-line based but is designed with simplicity and educational purposes in mind. The setup enables users to decompile APK files and manually inspect their underlying resources such as XML configurations, assets, and the overall project structure. The system comprises essential components like `apktool.jar`, `apktool.bat`, and the manual analysis process using a file explorer or an integrated APK analyzer tool. By leveraging APKTool's powerful decoding capabilities, users are able to retrieve and modify the original application's manifest files, resources, and `smali` code without the need for device-based extraction or complex environments. The tool ensures a lightweight, accessible reverse engineering experience suitable for students, cybersecurity researchers, and application developers who wish to audit apps for security, structure, or educational purposes.

The APK Analyzer workflow is intentionally designed to process files locally, ensuring user privacy and data security. As part of the usage guidelines, APK files are handled on the user's machine without being uploaded or stored remotely. Validation steps are recommended during the manual process to ensure only legitimate APK files are processed, reducing errors and improving the success rate of the decompilation. Future enhancements aim to automate the manual analysis phase, bringing a more streamlined experience through custom scripts and integrated GUI interfaces.

### 5.1 MODULE DESCRIPTION

The implementation of the APK Analyzer in this project is structured into three logical modules, each responsible for a specific phase of the reverse engineering workflow. This modular design supports the investigation of a suspicious APK file used in a cyber fraud case. The workflow starts with automated decompilation using `apktool.jar`, continues with user-friendly execution via a batch script, and concludes with a manual review of the decompiled structure to identify malicious JavaScript-based code. The architecture supports investigative clarity and offers a strong foundation for future enhancements like GUI automation for faster and broader analysis.

#### 1. `Apktool-jar` Module (Core Decompilation Logic)

This module is the core engine of the APK Analyzer. It uses `apktool.jar`, a widely adopted tool for decompiling Android application packages (APKs). Its primary task is to convert a raw APK into a structured, readable format to facilitate manual inspection.

**Key responsibilities include:**

- Decoding XML files such as AndroidManifest.xml into readable XML.
- Decoding the binary java class files.
- Extracting app resources like layouts, images, and assets from the res/ directory.
- Rebuilding the original directory structure of the application for easy navigation and analysis.

**2. APK Parsing Module ( using manifestparser.java)**

This lightweight module, implemented in Java, uses the xmlparser library to perform quick metadata extraction from the APK without full decompilation. It is useful for early-stage inspection or non-technical reviewers.

**Tasks performed:**

- Decoding the AndroidManifest.xml.
- Extracting basic metadata including:
  - App Name
  - Package Name
  - Version Code & Version Name
  - Declared Permissions
  - Launchable Activity

This module enables quick validation and classification of suspicious apps before deeper static code inspection.

**3. Apktool.bat Script (Trigger and Execution Handler)**

To streamline the reverse engineering process for analysts, a Windows batch script (apktool.bat) is provided. This script acts as the interface for triggering the decompilation process without requiring manual Java or command-line operations.

**Key functions include:**

- Configuring the Java runtime environment needed by apktool.jar.
- Accepting the APK file as input.
- Automatically executing the decompilation using apktool commands.

- Offering a simplified, one-click interface for cybersecurity analysts.

### **Module Workflow:**

The APK Analyzer tool follows a structured and investigative workflow to support cyber analysts in decompiling and examining suspicious mobile applications. This modular workflow enables effective detection of malicious components in hybrid apps.

#### **1. User Interaction:**

The cyber analyst obtains the suspicious APK file ("HAPPENING AROUND THE CITY") from a user's device and copies it to the investigation environment on a Windows machine.

#### **2. Decompile Execution:**

The analyst runs the apktool.bat script to trigger the decompilation process. This batch script:

- Automatically invokes apktool.jar.
- Decompiles the APK into a human-readable project structure.
- Reconstructs the full folder hierarchy, including manifest, assets, and resource files.

#### **3. APK Structure Analysis:**

The decompiled output is inspected to determine the app's framework. In this case, the app was identified as a hybrid mobile application built using the Ionic framework (JavaScript-based).

#### **4. Static Code Review:**

Analysts manually explore the decompiled folder structure:

- Review the AndroidManifest.xml for permission abuses.
- Filter .js files and search for suspicious keywords like sms, contacts, clipboard, storage.
- Perform static analysis to detect any malicious logic embedded in JavaScript files.

#### **5. Findings and Conclusion:**

Upon analysis, the app is confirmed to be fake and malicious, using deceptive content and hidden code to potentially steal user data.



## 6. Error Handling and Improvements:

If decompilation fails or unsupported APK formats are encountered, analysts address tool setup (Java, path issues).

Future plans include developing a GUI-based automation tool to highlight suspicious files, automate keyword filtering, and support less technical investigators.

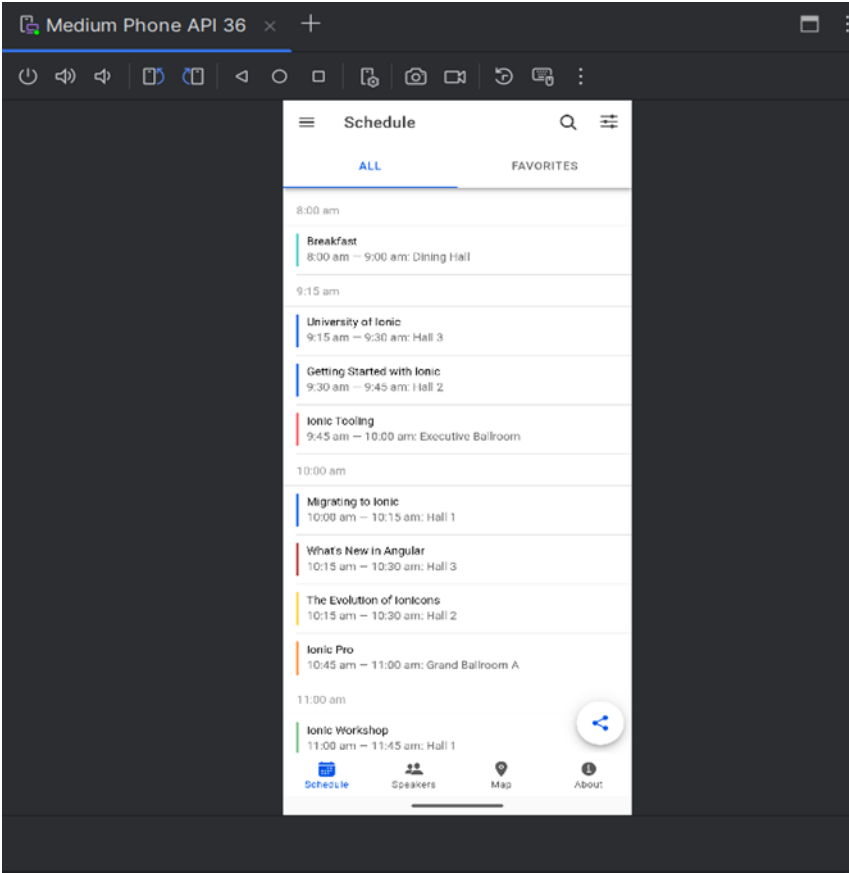
## 5.2 SCREEN SHOTS OF IMPLEMENTATION, RESULT

### CODE(Apktool.bat Code)

```
1 @echo off
2 setlocal
3 set BASENAME=apktool_
4 chcp 65001 2>nul >nul
5
6 set java_exe=java.exe
7
8 if defined JAVA_HOME (
9   set "java_exe=%JAVA_HOME%\bin\java.exe"
10 )
11
12 rem Find the highest version .jar available in the same directory as the script
13 setlocal EnableDelayedExpansion
14 pushd "%~dp0"
15 if exist apktool.jar (
16   set BASENAME=apktool
17   goto skipversioned
18 )
19 set max=0
20 for /f "tokens=1" delims=_.0" %A in ('dir /b /a-d %BASENAME%.jar') do if %A-8 gtr !max! set max=%A-nB
21 :skipversioned
22 popd
23 setlocal DisableDelayedExpansion
24
25 rem Find out if the commandline is a parameterless .jar or directory, for fast unpack/repack
26 if "%~1"=="" goto load
27 if not "%~2"=="" goto load
28 set ATTR=%~a1
29 if "%ATTR:~0,1%"=="d" (
30   rem Directory, rebuild
31   set fastCommand=b
32 )
```

```
19 set max=0
20 for /f "tokens=1" delims=_.0" %A in ('dir /b /a-d %BASENAME%.jar') do if %A-8 gtr !max! set max=%A-nB
21 :skipversioned
22 popd
23 setlocal DisableDelayedExpansion
24
25 rem Find out if the commandline is a parameterless .jar or directory, for fast unpack/repack
26 if "%~1"=="" goto load
27 if not "%~2"=="" goto load
28 set ATTR=%~a1
29 if "%ATTR:~0,1%"=="d" (
30   rem Directory, rebuild
31   set fastCommand=b
32 )
33 if "%ATTR:~0,1%"=="-" if "%~x1"=="apk" (
34   rem APK file, unpack
35   set fastCommand=d
36 )
37
38 :load
39 "%java_exe%" -jar -Xmx1024M -Duser.language=en -Dfile.encoding=UTF-8 -Djdk.util.zip.disableZip64ExtraFieldValidation=true -Djdk.nio.zipfs.allowDotZipEntry=true "%~dp0%BASENAME%.jar" %*
40
41 rem Pause when ran non interactively
42 for %i in (%cmdcmdline%) do if /i "%~i"==" /c" pause & exit /b
```

IMPLEMENTATION:

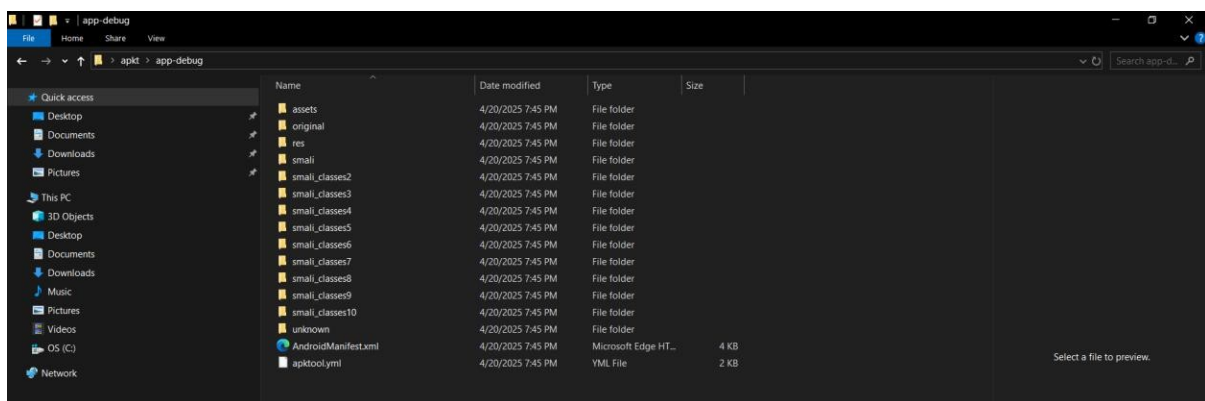


```
4/27/2025 1:33 PM BlueStacks Android 18.117 KB
C:\Windows\System32\cmd.exe - apktool d C:\Users\salomon\OneDrive\Desktop\apkt\app-debug.apk

Microsoft Windows [Version 10.0.19045.5737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\salomon\OneDrive\Desktop\apkt>apktool d /app-debug.apk
Input file (C:\app-debug.apk) was not found or was not readable.

C:\Users\salomon\OneDrive\Desktop\apkt>apktool d C:\Users\salomon\OneDrive\Desktop\apkt\app-debug.apk
I: Using Apktool 2.11.1 on app-debug.apk with 8 threads
I: Baksmaling classes.dex...
I: Baksmaling classes3.dex...
I: Loading resource table...
I: Baksmaling classes10.dex...
I: Baksmaling classes2.dex...
I: Baksmaling classes4.dex...
I: Baksmaling classes8.dex...
I: Baksmaling classes7.dex...
I: Decoding file-resources...
I: Loading resource table from file: C:\Users\salomon\AppData\Local\apktool\framework\1.apk
I: Baksmaling classes6.dex...
I: Baksmaling classes5.dex...
I: Baksmaling classes13.dex...
I: Baksmaling classes12.dex...
I: Baksmaling classes11.dex...
I: Baksmaling classes9.dex...
I: Decoding values */* XMLs...
I: Decoding AndroidManifest.xml with resources...
I: Regular manifest package...
I: Copying original files...
I: Copying assets...
```



## RESULTS:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android" android:compileSdkVersion="34" android:compileSdkVersionCodename="14" package="ionic.conference.app" platform
  <uses-permission android:name="android.permission.INTERNET"/>
  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
  <uses-permission android:name="android.permission.READ_CONTACTS"/>
  <uses-permission android:name="android.permission.WRITE_CONTACTS"/>
  <uses-permission android:name="android.permission.SEND_SMS"/>
  <uses-permission android:name="android.permission.READ_SMS"/>
  <uses-permission android:name="android.permission.VIBRATE"/>
  <permission android:name="ionic.conference.app.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION" android:protectionLevel="signature"/>
  <uses-permission android:name="ionic.conference.app.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION"/>
  <application android:allowBackup="true" android:appComponentFactory="androidx.core.app.CoreComponentFactory" android:debuggable="true" android:extractNativeLibs="true" andro
    <activity android:configChanges="keyboard|keyboardHidden|locale|orientation|screenLayout|screenSize|smallestScreenSize|uiMode" android:exported="true" android:label="@st
      <intent-filter>
        <action android:name="android.intent.action.MAIN"/>
        <category android:name="android.intent.category.LAUNCHER"/>
      </intent-filter>
    </activity>
    <provider android:authorities="ionic.conference.app.fileprovider" android:exported="false" android:grantUriPermissions="true" android:name="androidx.core.content.FilePro
      <meta-data android:name="android.support.FILE_PROVIDER_PATHS" android:resource="@xml/file_paths"/>
    </provider>
    <activity android:exported="false" android:name="com.google.android.gms.common.api.GoogleApiActivity" android:theme="@android:style/Theme.Translucent.NoTitleBar"/>
    <meta-data android:name="com.google.android.gms.version" android:value="@integer/google_play_services_version"/>
    <provider android:authorities="ionic.conference.app.androidx-startup" android:exported="false" android:name="androidx.startup.InitializationProvider">
      <meta-data android:name="androidx.emoji2.text.EmojiCompatInitializer" android:value="androidx.startup"/>
      <meta-data android:name="androidx.lifecycle.ProcessLifecycleInitializer" android:value="androidx.startup"/>
      <meta-data android:name="androidx.profileinstaller.ProfileInstallerInitializer" android:value="androidx.startup"/>
    </provider>
    <receiver android:directBootAware="false" android:enabled="true" android:exported="true" android:name="androidx.profileinstaller.ProfileInstallReceiver" android:permissi
      <intent-filter>
        <action android:name="androidx.profileinstaller.action.INSTALL_PROFILE"/>
      </intent-filter>
      <intent-filter>
        <action android:name="androidx.profileinstaller.action.SKIP_FILE"/>
      </intent-filter>
      <intent-filter>
        <action android:name="androidx.profileinstaller.action.CLEAR_PROFILE"/>
      </intent-filter>
    </receiver>
```

```
79 } || function(d2, b2) {
80   for (var p in b2) if (Object.prototype.hasOwnProperty.call(b2, p)) d2[p] = b2[p];
81 };
82 return extendStatics(d, b);
83 };
84 return function(d, b) {
85   extendStatics(d, b);
86   function __() {
87     this.constructor = d;
88   }
89   d.prototype = b === null ? Object.create(b) : (__.prototype = b.prototype, new __());
90 };
91 }();
92 var SmsRetrieverOriginal = (
93   /** @class */
94   function(_super) {
95     __extends(SmsRetrieverOriginal2, _super);
96     function SmsRetrieverOriginal2() {
97       return _super !== null && _super.apply(this, arguments) || this;
98     }
99     SmsRetrieverOriginal2.prototype.startWatching = function() {
100       return cordova(this, "startWatching", {}, arguments);
101     };
102     SmsRetrieverOriginal2.prototype.getAppHash = function() {
103       return cordova(this, "getAppHash", {}, arguments);
104     };
105     SmsRetrieverOriginal2.pluginName = "SmsRetriever";
106     SmsRetrieverOriginal2.plugin = "cordova-plugin-sms-retriever-manager";
107     SmsRetrieverOriginal2.pluginRef = "cordova.plugins.smsRetriever";
108     SmsRetrieverOriginal2.repo = "https://github.com/hanathashresh2712/ionic-native-sms-retriever-plugin-master";
109     SmsRetrieverOriginal2.install = 'ionic cordova plugin add cordova-plugin-sms-retriever-manager --variable PLAY_SERVICES_';
110     SmsRetrieverOriginal2.installVariables = ["PLAY_SERVICES_VERSION"];
111     SmsRetrieverOriginal2.platforms = ["Android"];
112     return SmsRetrieverOriginal2;
113   })(IonicNativePlugin)
114 );
```

## 5.3 COMPARISON TABLE AND GRAPH

### Comparison Table:

In the ever-evolving landscape of mobile security, APK analysis plays a crucial role in identifying potential vulnerabilities and malicious behavior within Android applications. Traditional APK analysis methods have served as basic tools for assessing the security of mobile apps, primarily focusing on metadata and visible permissions. However, with the rise of more sophisticated hybrid applications and increasingly advanced malware, a deeper, more comprehensive approach to APK analysis is necessary. The following comparison highlights the key differences between traditional APK analysis and a proposed, more robust APK analysis system, designed to offer an in-depth security investigation, better detection of hidden threats, and tailored analysis for modern app architectures, such as hybrid frameworks.

| Criteria | Traditional APK Analysis           | Proposed APK Analysis System                      |
|----------|------------------------------------|---------------------------------------------------|
| Purpose  | Quick assessment based on metadata | Deep security investigation and malware detection |

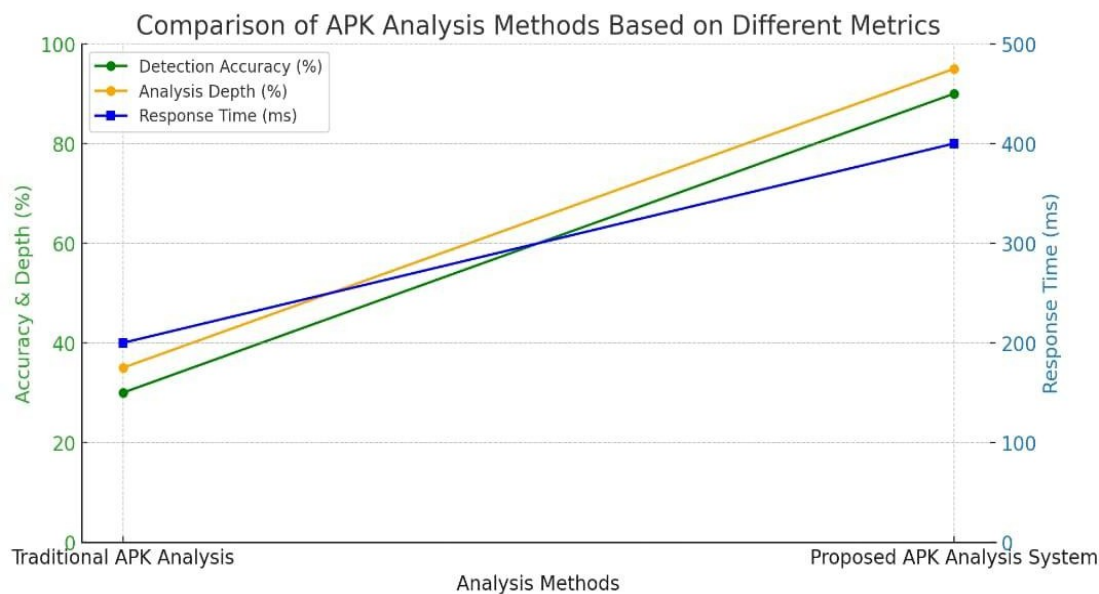
|                                     |                                                        |                                                                             |
|-------------------------------------|--------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>Tools Used</b>                   | Basic APK extractors, permission checkers              | Apktool, manual static analyzers, keyword search scripts                    |
| <b>Decompilation</b>                | Rarely decompiles APK files                            | Full APK decompilation using Apktool                                        |
| <b>Scope of Inspection</b>          | Limited to AndroidManifest.xml and visible permissions | Complete folder structure, JavaScript files, configuration files            |
| <b>Focus on Hybrid Applications</b> | Neglected; mainly targets native apps                  | Specially designed to analyze hybrid frameworks like Ionic                  |
| <b>Detection Techniques</b>         | Metadata review, permission list analysis              | Static code analysis, API tracing, keyword-based suspicious behavior search |
| <b>Handling of JavaScript Code</b>  | Not analyzed                                           | Thorough inspection of embedded JavaScript for hidden malicious logic       |
| <b>Analysis Depth</b>               | Shallow, metadata-focused                              | Deep, code-focused and behavior-focused                                     |
| <b>Output Generated</b>             | Basic information: app name, version, permissions      | Detailed security report with evidence (file paths, keywords, screenshots)  |
| <b>Skill Level Required</b>         | Low to moderate (easy for beginners)                   | Moderate to high (requires code reading and understanding)                  |
| <b>Vulnerability Detection</b>      | Very basic, often misses hidden threats                | Specifically targets hidden malicious behavior and data theft attempts      |
| <b>Environment Security</b>         | No special precautions                                 | Secure, isolated system environment maintained during analysis              |
| <b>User-Friendliness</b>            | High, but limited in information                       | Medium, but provides complete actionable insights                           |

|                                      |               |                                  |                                                                            |
|--------------------------------------|---------------|----------------------------------|----------------------------------------------------------------------------|
| <b>Customization and Scalability</b> |               | Difficult to customize or expand | Flexible for future upgrades like automation and dynamic behavior analysis |
| <b>Real-world Coverage</b>           | <b>Threat</b> | Poor, misses hybrid app threats  | Strong, covers real-world social engineering attack methods                |

Table 5.1 Comparison table

### Comparison Graph:

To effectively assess the security of Android applications, it is important to understand the differences between traditional and advanced APK analysis methods. Traditional analysis tools provide a quick overview by focusing on basic metadata and visible permissions, making them useful for initial assessments but limited in their ability to detect sophisticated threats. In contrast, the proposed APK analysis system offers a much deeper level of scrutiny, using techniques like full decompilation, static code analysis, and behavior tracking to uncover hidden vulnerabilities, particularly in hybrid applications. The following comparison graph illustrates these distinctions, highlighting how the new system improves threat detection and offers more detailed, actionable insights compared to traditional methods.



## CHAPTER 6

### CONCLUSION & FUTURE ENHANCEMENTS

The project on Reverse Engineering a Mobile App using APKTool addresses an important need in the field of mobile security. The solution developed provides a lightweight and efficient method for decompiling and analyzing mobile applications, making the process more accessible for students, researchers, and beginner security analysts. Reverse engineering typically requires specialized tools and considerable expertise, often acting as a barrier for newcomers in the field. By focusing on APKTool, a powerful yet accessible tool, this project simplified the task of inspecting and understanding mobile applications at the resource and code level.

The approach adopted a modular setup, which includes apktool.jar for core decompilation, a streamlined apktool.bat trigger script, and a manual inspection phase. This allows users to decompile APKs easily and explore essential components such as AndroidManifest.xml, resources, and smali code. The manual analysis of decompiled resources provides valuable insights into app permissions, activities, services, and potential vulnerabilities without requiring the application to run or perform dynamic testing.

Throughout the project, emphasis was placed on ensuring simplicity, data privacy, and transparency. This made the solution an ideal learning tool for those entering the field of mobile security and reverse engineering. Overall, the project successfully met its objectives by offering a lightweight, non-invasive, and educational approach to understanding mobile applications through static analysis. This has laid the groundwork for future enhancements and deeper security evaluation.

#### Future Enhancements

1. **Automated Analysis Scripts:** Develop scripts to automatically extract key information such as permissions, activities, services, and hardcoded URLs from the decompiled output.
2. **Graphical User Interface (GUI):** Build a simple desktop or web-based GUI to allow drag-and-drop APK decompilation, eliminating the need for command-line interaction.
3. **Threat Detection Indicators:** Integrate basic static analysis features to flag suspicious code patterns, dangerous permissions, or signs of malware behavior.

4. **Batch Processing:** Enable the tool to handle multiple APK files simultaneously for comparative analysis or bulk inspection.

5. **Certificate and Signature Validation:** Incorporate functionality to extract and verify APK signing certificates to detect tampered or repackaged applications.

6. **Integration with Other Tools:** Allow interoperability with tools like **VirusTotal API**, **JADX**, or **MobSF** to create a more comprehensive security analysis pipeline.

7. **Mobile Version:** Develop a lightweight Android-based tool for on-device APK analysis, aimed at mobile security testing and educational demonstrations.

These enhancements would not only make the APK Analyzer more user-friendly and powerful, but also extend its reach to professional cybersecurity assessments, penetration testing, and academic research in mobile application security.



## References:

1. Votipka, Daniel, Mary Nicole Punzalan, Seth M. Rabin, Yla Tausczik, and Michelle L. Mazurek. "An Investigation of Online Reverse Engineering Community Discussions in the Context of Ghidra." *Proceedings of the 2021 ACM Conference on Computer and Communications Security (CCS)*, Vol. 2021, 2021.
2. Tamilkodi, R., V. Bala Sankar, Shrija Madhu, L. Revathi, V. Sai Guna Srikar, and Ernagula Ajay. "Exploring IoT Device Vulnerabilities Through Malware Analysis and Reverse Engineering." *International Journal of Engineering Research & Technology (IJERT)*, Vol. 13, 2024.
3. Cao, Kevin, and Kevin Leach. "Revisiting Deep Learning for Variable Type Recovery." *Proceedings of the IEEE Symposium on Security and Privacy*, Vol. 2023, 2023.
4. Poudyal, Subash, and Dipankar Dasgupta. "AI-Powered Ransomware Detection Framework." *Journal of Cybersecurity and Privacy*, Vol. 2021, 2021.
5. Zhang, Runze, Ying Cao, Ruigang Liang, Peiwei Hu, and Kai Chen. "Optimizing Decompiler Output by Eliminating Redundant Data Flow in Self-Recursive Inlining." *Proceedings of the 2024 Annual Computer Security Applications Conference (ACSAC)*, Vol. 2024, 2024.
6. Rhee, Junghwan "John", Fei Zuo, Cody Tompkins, Jeehyun Oh, and Ying Ryn Choe. "GrizzlyBay: Retroactive SBOM with Automated Identification for Legacy Binaries." *Proceedings of the 2024 Annual Computer Security Applications Conference (ACSAC)*, Vol. 2024, 2024.
7. Akram, Junaid, Zhendong Shi, Majid Mumtaz, and Ping Luo. "DroidCC: A Scalable Clone Detection Approach for Android Applications to Detect Similarity at Source Code Level." *IEEE Access*, Vol. 6, 2018.
8. Yulianto, Semi, and Gerard Nathaniel Cac Ngo. "Automated Threat Hunting, Detection, and Threat Actor Profiling Using TIRA." *International Journal of Computer Applications*, Vol. 2024, 2024.
9. Akram, Bio, and Dion Ogi. "The Making of Indicator of Compromise using Malware Reverse Engineering Techniques." *Journal of Information Security Research*, Vol. 2021, 2021.
10. Bhardwaj, Vivek, Vinay Kukreja, Chetan Sharma, Isha Kansal, and Renu Popali. "Reverse Engineering—A Method for Analyzing Malicious Code Behavior." *International Journal of Scientific & Technology Research*, Vol. 11, 2022.

11. Yan, Jihong, Huaqiu Ding, Jinsong Zhang, and Pengxiang Wang. "Point Cloud Data Fitting-Based Reverse Engineering Technique for Digital Model Updating of Assembly Structure." *Advanced Engineering Informatics*, Vol. 50, 2021.
12. Brdjanin, Drazen, Dragana Vukovic, Goran Banjac, Aleksander Kelec, Igor Dujlovic, and Nikola Obradovic. "REDBUL: An Online System for Reverse Engineering of Relational Databases." *Date of Publication: 11 September 2020*.
13. Widiyasono, Nur, Siti Rahayu Selamat, Randi Rizal, Abdul Fidayan, Sri Rochani Mulyani, and Slamet Risnato. "Advanced Malware Analysis Methods: Behaviour-Based Detection and Reverse Engineering." *Date of Publication: 06 February 2025*.
14. Siala, Hanan Abdulwahab. "Enhancing Model-Driven Reverse Engineering Using Machine Learning." *Date of Publication: 20 June 2024*.
15. Dharmalingam, Balakrishnan, Anyi Liu, Subra Ganesan, and Sankardas Roy. "FineObfuscator: Defeating Reverse Engineering Attacks with Context-sensitive and Cost-efficient Obfuscation for Android Apps." *Date of Publication: 27 July 2022*.